BASIC GIS MAP ANALYZER

TEAM MEMBERS:

1. AYUSH KUMAR SINGH
   (RA2511003010309)
2. NAMAN SHARMA
   (RA2511003010323)
3. VIKRANT SINGH
   (RA2511003010347)
4. ARITRA DAS
   (RA2511003010294)

COURSE: OODP
(21CSC101T)

DEPARTMENT: BTECH/CSE CORE

(2025-2026)

INTRODUCTION

- This project is a GIS-based geometric computation system developed using C++ and Python (Flask). It enables the creation of points, lines, and polygons, and computes essential properties such as distance and area. The system uses a CSV file for data exchange and presents the results through a structured web dashboard. It demonstrates the integration of backend computation with web-based visualization while applying core Object-Oriented Programming principles.

- Provides dynamic data updates and real-time result visualization

- Designed with a modular architecture for ease of maintenance and scalability

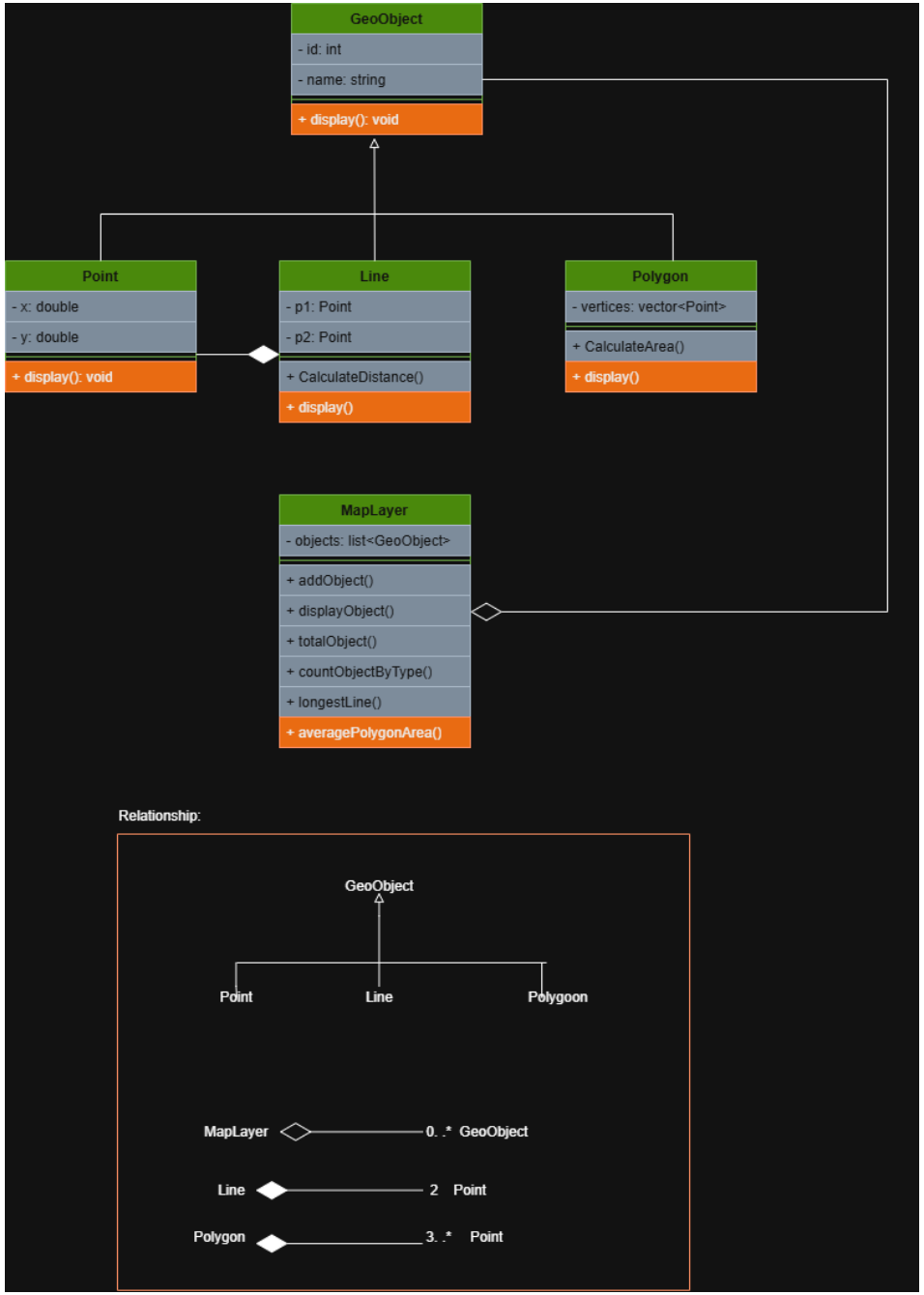
PROBLEM STATEMENT

- Design and implement a basic GIS analyzer that models spatial entities using OOP in C++, computes key metrics (line distance and polygon area), exports results to CSV, and visualizes the output through a Flask-based web dashboard.

SYSTEM DESIGN

- The system follows a two-tier architecture: a C++ backend for GIS computation and a Flask frontend for visualization. The backend models spatial entities (Point, Line, Polygon) using OOP, computes line distance and polygon area, and exports results to a CSV file. The frontend reads this CSV and renders summary, data tables, and statistics pages.
- Modules
- C++ GIS Engine (**`cpp/`**)
  - Classes: `GeoObject`, `Point`, `Line`, `Polygon`, `MapLayer`
  - Responsibilities: object modeling, computation, CSV export
- Flask Web Dashboard (**`web/`**)
  - Reads `cpp/data.csv`
  - Routes: `/home`, `/data`, `/stats`
  - Displays computed results in a user-friendly format
  - Data Flow
    User runs C++ program → metrics computed → results saved to CSV → Flask app loads CSV → dashboard displays outputs.

UML DIAGRAM->


GeoObject
- id: int
- name: string
+ display(): void
Point
- x: double
- y: double
+ display(): void
Line
- p1: Point
- p2: Point
+ CalculateDistance()
+ display()
Polygon
- vertices: vector<Point>
+ CalculateArea()
+ display()
MapLayer
- objects: list<GeoObject>
+ addObject()
+ displayObject()
+ totalObject()
+ countObjectByType()
+ longestLine()
+ averagePolygonArea()
Relationship:
GeoObject
Point
Line
Polygoon
MapLayer
0. .* GeoObject
Line
2 Point
Polygon
3. .* Point

EXPLANATION OF OOPS CONCEPT USED

- Inheritance
  Common GIS entities (`Point`, `Line`, `Polygon`) derive from a base class `GeoObject`, enabling shared structure and behavior.
- Polymorphism
  The base class defines a virtual method `display()`, and `MapLayer` stores `GeoObject*`. This allows dynamic dispatch so each object shows its own details at runtime.
- Encapsulation
  Geometry data members are kept `private` or `protected`, and accessed through public methods, ensuring controlled access and data integrity.
- Abstraction
  The `GeoObject` class defines a common interface for all spatial entities, hiding implementation details while exposing essential behaviors.

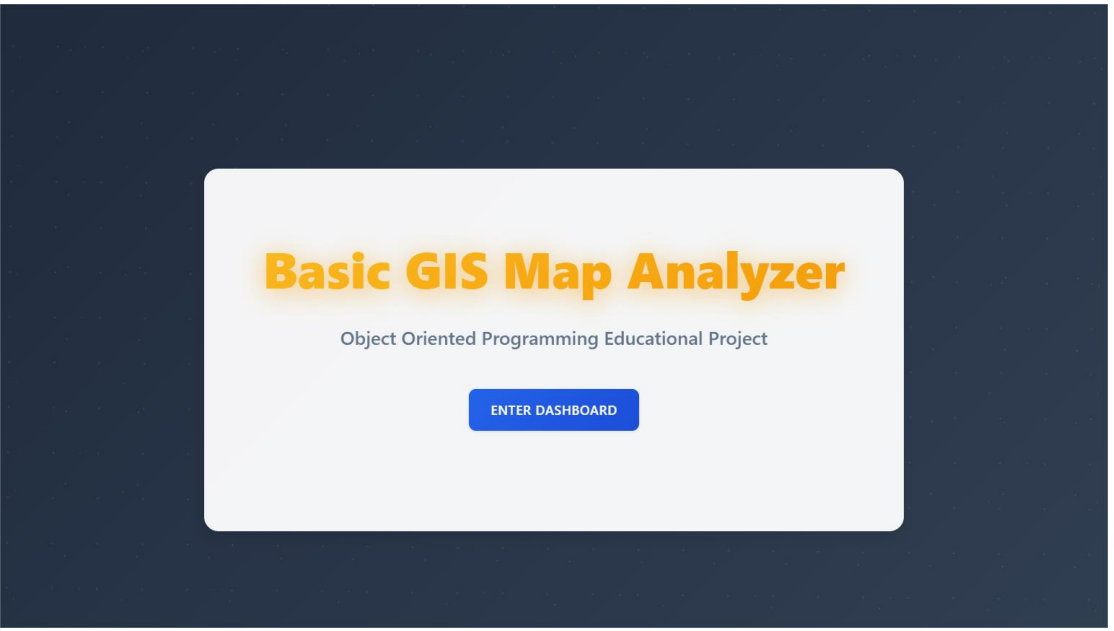
here's a small code snippet showing Inheritance + Polymorphism in our project:

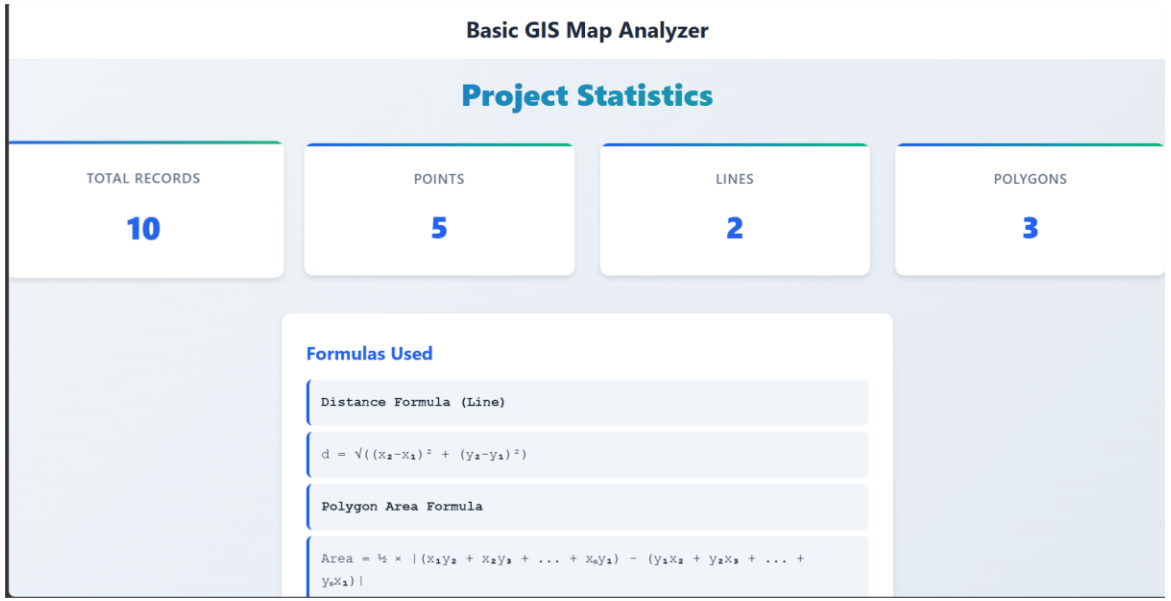
app.py | uml.pdf U | projectreport.pdf U | script.js M

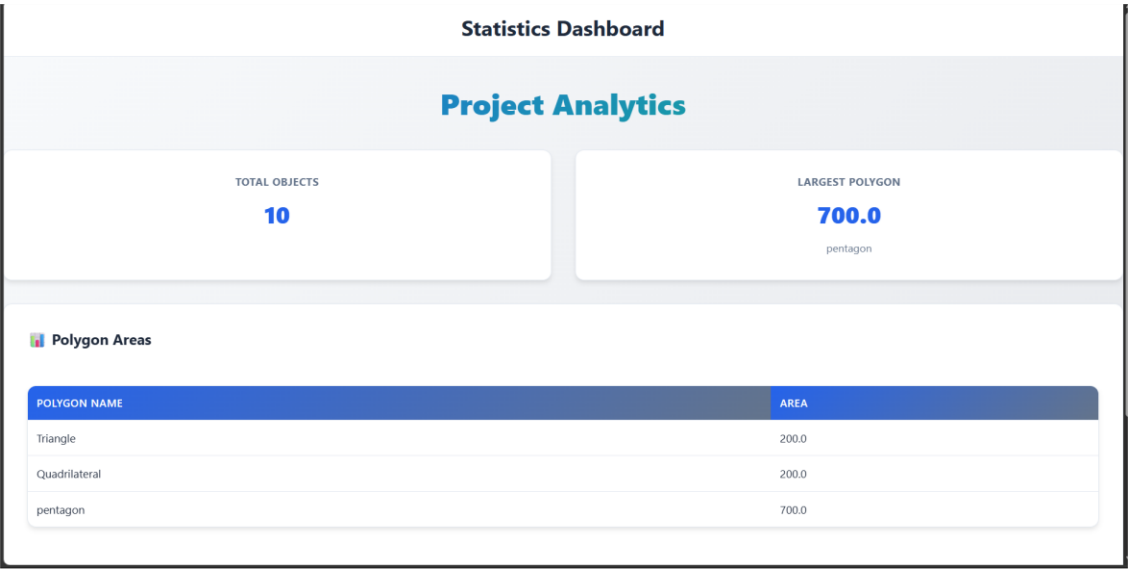
cpp > classes.h > ...

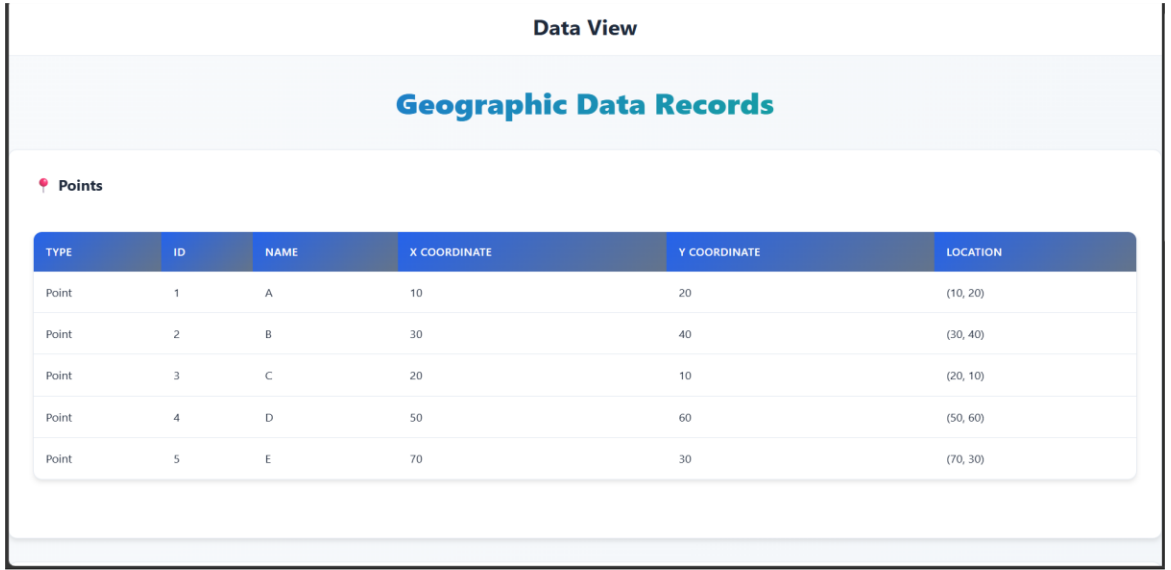
```cpp
// Base Class
class GeoObject {
protected:
    int id;
    string name;

public:
    GeoObject(int id, string name);
    virtual void display() = 0;
    int getID();
    string getName();
};

// Derived Class FOR PONTS
class Point : public GeoObject {
private:
    double x;
    double y;

public:
    Point(int id, string name, double x, double y);

    double getX();
    double getY();

    void display();
};
```

SCREENSHOTS









CONCLUSION

- This project demonstrates how OOP principles can be applied to real GIS-style problems. Spatial objects are modeled in C++, key metrics are computed accurately, and results are exported to CSV for easy integration. The Flask dashboard provides a clear visualization layer, making the system practical, modular, and educational for understanding both programming concepts and basic spatial analysis.